1 Short questions

1. Piping is the mechanism in Unix that allows a process to redirect its output to a file, instead of the standard output.
   Your answer(True/False) :

2. Name the environment variable that stores the list of directories used by the shell to search for binary files to be executed
   Your answer :

3. The standard error stream *stderr* can be used by the Unix system calls *read()* and *write()*.
   Your answer(True/False) :

4. The function **fgets()** reads characters from a stream into an array and adds a new line character at the end.
   Your answer(True/False) :

5. What is the difference between unnamed pipes and FIFOs?
   Your answer :

6. What is the difference between sockets and FIFOs?
   Your answer :

7. A process can block or ignore a SIGKILL signal.
   Your answer(True/False) :

8. When a process **P** creates a child process **P1**, then creates another child process **P2**, the pid of **P1** is greater than the pid of **P2**.
   Your answer :

9. System calls **read()** and **write()** use buffering when dealing with files.
   Your answer(True/False) :

10. How many values does the system call **fork()** return if successful?
    Your answer :

11. How many values does the system call **fork()** return if it fails?
    Your answer :

12. Every user process must have a PID and a PPID, what happens to the PPID of a child process when its parent dies while the child is still alive?
    Your answer :

13. When using sockets in the client/server context, a client program only needs to know the IP of the server's machine in order to get connected to the server.
    Your answer (True/False):

14. A socket has a name that exists in the unix file system.
    Your answer (True/False):

15. We can read from and write to the same FIFO.
    Your answer (True/False):

2 Tracing(6 questions) : 6 marks each

1. Consider the program

```
int main(int argc, char *argv[]){
 int fd;
 char buffer1[10] = "One ";
 char buffer2[10] = "Two ";

 fd = open("file.txt", O_WRONLY|O_CREAT|O_TRUNC);
 write(fd, buffer1, strlen(buffer1));
 write(STDOUT_FILENO, buffer2, strlen(buffer2));
// STDOUT_FILENO is the screen file descriptor
 dup2(fd, STDOUT_FILENO);
 write(fd, buffer1, strlen(buffer1));
 write(STDOUT_FILENO, buffer2, strlen(buffer2));
 close(fd);
}
```

   - What will appear on the screen?
     Your answer(3 marks) :

   - What will the file **file.txt** contain?
     Your answer(3 marks) :

2. How many "**Hello**"s will appear on the screen when this program is executed?

```
int main(int argc, char *argv[]){
  fork();
  printf("Hello\n");
  fork();
  printf("Hello\n");
}
```

Your answer :

3. How many "**Hello**"s will appear on the screen when this program is executed?

```
int main(int argc, char *argv[]){
   int i;

   for(i=1; i<=3; i++){
     fork();
     printf("Hello\n");
   }
   printf("Hello\n");
}
```

Your answer :

4. What will be printed when the program below runs?

```
int main(int argc, char *argv[]){
  printf("A ");
  fork();
  printf("B\n");
  fork();
  pause();
  printf("Bye\n");
}
```

Your answer :

5. What will be printed by the following program

```
void action(){
  fprintf(stderr, " Thursday ");
  exit(1);
}
int main(){
  pid_t pid, st;

  if(!(pid=fork())){
    signal(SIGALRM, action);
    alarm(1);
    while(1)
      sleep(1);
    fprintf(stderr, "exam ");
  }else{
```

```
    fprintf(stderr, "Final");
    wait(&st);
    fprintf(stderr, "%d\n", WEXITSTATUS(st));
  }
}
Hint: WEXITSTATUS(st) extracts exit status from st
```

Your answer :

6. What will be printed by the following program

```
void action(){
  fprintf(stderr, "OK ");
  exit(0);
}
int main(){
  pid_t pid;

  if(!(pid=fork())){
    signal(SIGUSR1, action);
    pause();
    fprintf(stderr, "First ");
  }else{
    sleep(1);
    fprintf(stderr, "Thursday ");
    kill(pid, SIGUSR1);
   }
  sleep(1);
  fprintf(stderr, " Next ");
}
```

Your answer :

7. What will appear on the screen when the program below executes?

```
int main(int argc, char *argv[]){
  int fd;
  char name1[15] = "Bonjour\n";
  char name2[15] = "Bonsoir\n";

  fd = open("file.txt", O_WRONLY|O_CREAT|O_TRUNC);
  write(fd, name1, 8);
  close(fd);
  fd = open("file.txt", O_RDONLY);
  dup2(fd, 0);
  read(0, name2, 8);
```

```
  write(1, name2, 8);
}
```

8. The program below is a simple client that reads a single message from a remote server. Explain what crucial information is missing for this client in order for it to be working correctly?

```
int main(int argc, char *argv[]){
  char message[100];
  int n, server;
  struct sockaddr_in servAdd;

  server=socket(AF_INET, SOCK_STREAM, 0);
  servAdd.sin_family = AF_INET;
  inet_pton(AF_INET, argv[1], &servAdd.sin_addr);
  connect(server, (struct sockaddr *) &servAdd, sizeof(servAdd));
  n=read(server, message, 100);
  write(1, message, n);
  exit(0);
}
```